

NATURAL LANGUAGE PROCESSING APPLICATIONS

Mid-Semester One-Stop Master Guide

Course: AIMLCZG519

Coverage: Contact Sessions 1 to 7

Based on lecture slides, course handout, full class transcripts, and sample examination papers

Lokesh..

Fiendly Disclaimer

This document is just to help prepare for the mid-sem and consolidate your understanding of the concepts, but it does not guarantee anything regarding the actual exam questions. Use it only if you do not have enough time to prepare properly.

Purpose & How to Use This Guide

A full-coverage, exam-oriented guide containing every lecture topic, instructor emphasis, formula, solved numerical, real-life example, comparison table, and a fully solved sample mid-semester paper. Includes visual architectures, simplified explanations and step-by-step math breakdowns.

- **First pass:** Read Chapters 1 to 7 in order. Pay attention to the **Concept Simplified** boxes and **Architectural Diagrams**.
- **Numerical pass:** Rework every **Solved Example** box. Use the **Formula Explained** boxes to understand the "why" behind the math.
- **Final pass:** Use the Formula Sheet, comparison tables, the fully solved paper, and the 6-mark practice questions in Chapter 10.
- **For 6-mark answers:** write a definition, architecture or steps, one example, advantages, limitations, evaluation, and a short conclusion.
- **For design questions:** state assumptions, identify data sources, propose components, show the flow, discuss updating and governance, and name suitable metrics.

Latest instructor exam indication

The June 6 transcript states that the mid-semester examination is expected to contain five questions totalling 30 marks, usually around 6 marks each. Expected areas include spell and grammar checking, sentiment analysis, question answering, conversational AI, RAG applications, and smaller questions on privacy and ethics.

Contents

1 Course Scope, NLP Application Landscape, and Evaluation	4
1.1 Course objectives and official scope	4
1.2 Why NLP applications are difficult	4
1.3 Application pipeline & Hybrid Design	4
1.4 Evaluation is application-specific	5
2 Spell Checking and Grammar Checking	6
2.1 Definitions and error categories	6
2.2 Dictionary-based techniques	6
2.3 Candidate generation & Instructor Emphasis	6
2.4 Noisy-channel model	7
2.5 Channel model and confusion matrices	7
2.6 Context-sensitive and real-word correction	8
2.7 Statistical, ML, transformer, and LLM approaches	8
2.8 Evaluation	8
3 Retrieval-Augmented Generation (RAG)	10
3.1 What RAG is and why it is needed	10
3.2 RAG Variants Comparison	10
3.3 Retrieval Methods and Cosine Similarity	11
3.4 RAG Evaluation (RAGAS-style Metrics)	11
4 Question Answering Systems	12
4.1 Question Types and Evolution	12
4.2 Retrieval and Ranking Metrics	12
5 Conversational AI Systems	14
5.1 Core Components and Evolution	14
5.2 Intent classification and slot filling	14
6 Sentiment Analysis and Opinion Mining	15
6.1 Turney algorithm and PMI	15
7 Privacy, Ethics, Safety, and Governance	15
7.1 Model fairness metrics	15
7.2 Tools and Case Studies	15
8 Master Formula Sheet and Numerical Templates	17
9 Fully Solved Previous Mid-Semester Paper	17
10 High-Probability Practice Questions with Solutions (6-Mark Format)	19
11 Final Revision Checklist	20

1 Course Scope, NLP Application Landscape, and Evaluation

Session 1 establishes what NLP applications do, why they are hard, and how to evaluate them.

1.1 Course objectives and official scope

The course focuses on identifying algorithms for real-world NLP applications, using NLP tools, applying modern methods to applications such as machine translation and information extraction, and evaluating implementation approaches together with ethical considerations.

Application area	Core task	Typical output	Common risk
Spell & grammar	Detect and correct errors	Corrected word/sentence	Wrong correction, dialect bias
Question answering	Return a direct answer	Fact, span, or answer	Hallucination, poor ranking
Conversational AI	Maintain a multi-turn dialogue	Response plus action	Context loss, unsafe action
RAG & knowledge graphs	Ground an LLM in evidence	Answer with support	Bad retrieval, stale evidence
Machine translation	Translate meaning	Target-language text	Ambiguity, terminology errors
Information extraction	Extract entities, relations	Structured records	Boundary and linking errors
Sentiment analysis	Detect opinions, polarity	Label or score	Sarcasm, domain drift, bias
Privacy and ethics	Control data/model risks	Policies, safeguards	Leakage, discrimination

1.2 Why NLP applications are difficult

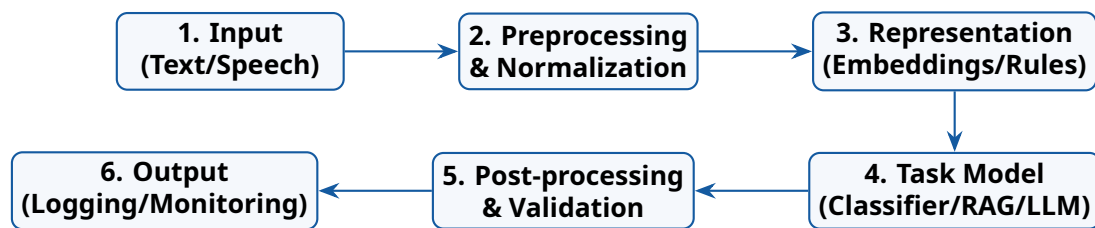
Language is ambiguous at lexical, syntactic, semantic, discourse, and pragmatic levels. The same surface form can express different intentions depending on context. "Book" can be a noun or an action. Real applications must handle noisy input, spelling variations, slang, multilingual text, code-mixing, domain terminology, and incomplete questions.

Concept Simplified: Why High Accuracy isn't Enough

High benchmark accuracy on a test dataset does not guarantee a safe, real-world deployment. If an AI has 99% accuracy but the 1% error causes a massive data leak or highly offensive output, it fails. Real systems must balance latency, cost, fairness, privacy, explainability, and fallback design.

1.3 Application pipeline & Hybrid Design

Modern production systems are usually hybrid.



Hybrid system design principle: A production NLP application should use the simplest reliable component for each subproblem. A banking assistant might use deterministic rules for compliance messages, NER for account entities, a classifier for intent, RAG for policy questions, an LLM for explanation, and an agent only when a tool call is required.

1.4 Evaluation is application-specific

Application	Primary automatic metrics	Human evaluation focus
Spell and grammar	Precision, recall, F0.5, ERRANT	Usefulness and false-positive annoyance
QA and retrieval	EM, token F1, P@k, R@k, MRR, MAP, nDCG	Correctness, completeness, conciseness
RAG	Context precision/recall, faithfulness, answer relevance	Groundedness and citation quality
Conversational AI	Intent accuracy, SER, task success, response quality	Naturalness, recovery, user satisfaction
Sentiment	Accuracy, macro/micro F1, ROC-AUC	Aspect correctness and domain validity
Translation	BLEU, chrF, COMET	Meaning preservation and fluency
Information extraction	Entity/relation precision, recall, F1	Boundary and semantic correctness
Privacy and ethics	Fairness gaps, leakage tests, toxicity, calibration	Acceptable risk and accountability

2 Spell Checking and Grammar Checking

From edit distance and noisy-channel correction to transformers, hybrid tools, and grammatical-error evaluation.

2.1 Definitions and error categories

A spell checker detects misspelled words and proposes corrections. A grammar checker detects syntactic, semantic, agreement, punctuation, and style problems.

Category	Meaning	Example	Why it is difficult
Non-word error	Token absent from dictionary	recieve → receive	Candidate generation is usually enough
Real-word error	Token is valid but wrong in context	their car → there car	Requires context language modelling
Syntactic error	Violates grammatical structure	She go to school	Requires agreement rules or parsing
Semantic anomaly	Grammatical but implausible	colourless green ideas	Needs semantics and world knowledge
Punctuation/style	Incorrect punctuation/style	missing comma	Rules depend on genre/preference

2.2 Dictionary-based techniques

- **Levenshtein distance:** Minimum insertions, deletions, substitutions. $O(mn)$.
- **Damerau-Levenshtein:** Also allows adjacent transposition (e.g., ie → ei).
- **Trie / BK-tree / Character k-gram index:** Used for fast, sublinear candidate generation.
- **Soundex/phonetic coding:** Maps similar pronunciations to a common code. Linear in token length. Best for names and phonetic errors.

Solved Example 2.1: Levenshtein distance

Problem: Transform "receiv" into "receive".

Solution: The first six characters ("r-e-c-e-i-v") already match. We only need to insert an "e" at the end.

Operations: 1 Insertion.

Result: Levenshtein distance is 1.

Note: For "recieved" → "received", ordinary Levenshtein counts 2 substitutions, while Damerau-Levenshtein counts 1 adjacent transposition. In an exam, state which distance definition you are using.

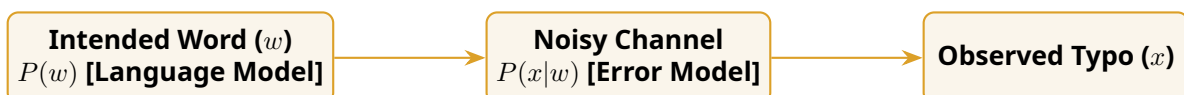
2.3 Candidate generation & Instructor Emphasis

- Check each dictionary word and retain words within edit distance k .
- Generate all strings within one or two edits and intersect them with the dictionary.

- Use character k-gram overlap and rank candidates using Jaccard similarity.
- **Instructor emphasis:** Roughly 80% of typing errors are within edit distance 1, and almost all common errors are within distance 2. For exam numericals, non-word noisy-channel calculations are more likely than combinatorial real-word correction.

2.4 Noisy-channel model

The observed misspelling (x) is treated as a corrupted version of an intended word (w). Bayes rule ranks each candidate by how common the intended word is and how plausible the typing error is.



Formula Explained: Noisy Channel Model: $\hat{w} = \arg \max_w P(x|w) \times P(w)$

We want to find the intended word ($\hat{w}$) given the typo we see (x).

- $P(w)$ is the **Language Model (Prior)**: How common is this word in English? "receive" is common, "deceive" is rare.
- $P(x|w)$ is the **Error Model (Likelihood)**: If the user meant to type "receive" (w), what are the odds their fingers slipped and typed "recieve" (x)?

We multiply these two together. The candidate word with the highest product wins. We don't divide by $P(x)$ because $P(x)$ is the same for all candidates.

Solved Example 2.2: Correcting "recieved" using Noisy Channel

Candidate (w)	$P(w)$ [Prior]	$P(x w)$ [Likelihood]	Product ($P(x w) \times P(w)$)
received	0.0000247	0.00040000	9.880e-09 (Highest)
receive	0.0000197	0.00020000	3.940e-09
receiver	0.0000091	0.00006000	5.460e-10
deceived	0.0000043	0.00003000	1.290e-10

Conclusion: The largest product is 9.88×10^{-9} for "received". Therefore the corrected word is "received".

2.5 Channel model and confusion matrices

A confusion matrix records how often one character operation produces another.

Formula Explained: Add-One (Laplace) Smoothing

$$P(x|w) = \frac{\text{count(error_pattern)}+1}{\text{count(intended_context)}+|A|}$$

If a specific typo has never happened in our training data, its probability $P(x|w)$ becomes 0. If we multiply by 0, the whole candidate score becomes 0, ruining the math.

Fix: We add 1 to the numerator (so it's never zero) and we add $|A|$ (the size of the alphabet,

usually 26) to the denominator to keep the probabilities balanced.

Solved Example 2.3: Add-one smoothing

Suppose the substitution $q \rightarrow a$ never appeared, q occurred 50 times as the intended character, and the alphabet has 26 letters.

$$P(a \text{ observed} \mid q \text{ intended}) = (0 + 1)/(50 + 26) = 1/76 = \mathbf{0.01316}.$$

Without smoothing the probability would be zero.

2.6 Context-sensitive and real-word correction

$P(w_1 \dots w_n) \approx P(w_1) \times \prod_{i=2}^n P(w_i \mid w_{i-1})$ Bigram language model used to compare sentence candidates. For "a stellar and versatile actress", both actress and across can be close spelling candidates. The bigram probabilities $P(\text{actress} \mid \text{versatile})$ and $P(\text{across} \mid \text{versatile})$, plus the next-word probability, usually favour actress. A real-word checker keeps the original word as one candidate and compares complete sentence probabilities.

2.7 Statistical, ML, transformer, and LLM approaches

Approach	Mechanism	Strength	Limitation
N-gram/noisy channel	Ranks candidates by corpus freq	Interpretable, cheap	Limited long-range context
CRF/SVM	Classifies token errors using features	Fast and controllable	Feature engineering
RNN/LSTM	Models character/word sequences	Captures long dependencies	Sequential and slower
BERT/RoBERTa	Detects unlikely tokens via context	Strong detection	Needs a separate corrector
T5/BART/mT5	Maps erroneous to corrected sent.	Fluent end-to-end correction	Can over-correct
LLM prompting	Zero/few-shot correction	Flexible	Cost, latency, privacy
Hybrid ensemble	Combines rules, stats, and neural	Best precision and coverage	Engineering complexity

2.8 Evaluation

Precision = correct suggestions/all suggestions

Recall = detected true errors/all actual errors

Formula Explained: F_β Score for Grammar Correction

$$F_\beta = (1 + \beta^2) \frac{P \times R}{\beta^2 P + R}$$

Grammatical error correction commonly uses $F_{0.5}$ (where $\beta = 0.5$).

Why? A $\beta < 1$ weights **Precision** heavier than Recall. In a grammar checker, false corrections (changing a correct word to a wrong one) annoy users massively. It is better to miss

an error (lower recall) than to confidently make a wrong suggestion (low precision).

Solved Example 2.4: $F_{0.5}$ Score Calculation

A checker makes 80 suggestions, 60 are correct. The corpus has 100 actual errors.

$$\text{Precision}(P) = 60/80 = 0.75$$

$$\text{Recall}(R) = 60/100 = 0.60$$

$$F_{0.5} = (1 + 0.5^2) \frac{0.75 \times 0.60}{(0.5^2 \times 0.75) + 0.60} = 1.25 \times \frac{0.45}{0.1875 + 0.60} = \frac{0.5625}{0.7875} = \mathbf{0.714}$$

3 Retrieval-Augmented Generation (RAG)

3.1 What RAG is and why it is needed

RAG retrieves external evidence at inference time and places it in the LLM prompt.

- Reduces hallucination by grounding claims in evidence.
- Overcomes the model knowledge cutoff (no need to retrain).
- Supports citations and traceability.
- Adds private enterprise knowledge without exposing it during base-model training.

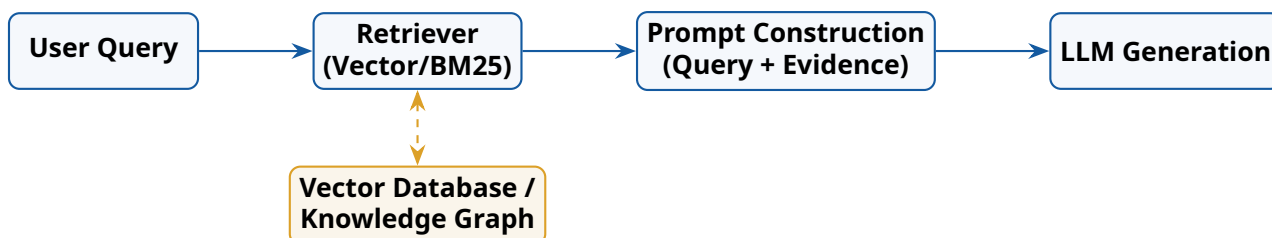
3.2 RAG Variants Comparison

Dimension	Naive RAG	Graph RAG	Agentic RAG	Multimodal RAG
Retrieval	One-shot tor/sparse	vec- Graph traversal	Dynamic multi-step	Cross-modal
Reasoning	Limited	Relationship-driven	Planning and tools	Fusion across types
Latency	Medium	Medium to high	High	High
Cost	Moderate	Higher setup	Highest runtime	High storage
Best fit	Document QA	Connected knowl- edge	Complex tasks	Visual documents

Concept Simplified: Graph RAG vs Agentic RAG

Graph RAG: Converts data into Nodes (Employee, Project) and Edges (WORKS_ON). If you ask "Who led Project X?", it traces the explicit graph path instead of relying on flat vector similarity. Great for interconnected data.

Agentic RAG: The LLM acts as a "manager." It gets a query, breaks it into sub-tasks, decides to use a search tool, looks at the results, decides if it needs to search again, and then synthesizes the answer. Great for complex, multi-step reasoning.



3.3 Retrieval Methods and Cosine Similarity

Formula Explained: Cosine Similarity: $\cos(q, d) = (q \cdot d) / (||q|| \times ||d||)$

Used in Dense Vector Retrieval. It measures the angle between two vectors (the query embedding and the document embedding). A score of 1 means they point in the exact same semantic direction (highly relevant).

Solved Example 3.1: Cosine Similarity

Let query $q = [1, 1]$ and two document chunks $d_1 = [2, 1]$, $d_2 = [0, 3]$.

Numerator $(q \cdot d_1) = (1 \times 2) + (1 \times 1) = 3$

Denominator $||q|| = \sqrt{1^2 + 1^2} = \sqrt{2}$. $||d_1|| = \sqrt{2^2 + 1^2} = \sqrt{5}$.

$\cos(q, d_1) = 3 / (\sqrt{2} \times \sqrt{5}) = 3 / \sqrt{10} = \mathbf{0.949}$.

$\cos(q, d_2) = 3 / (\sqrt{2} \times 3) = \mathbf{0.707}$.

Conclusion: d_1 ranks first because $0.949 > 0.707$.

3.4 RAG Evaluation (RAGAS-style Metrics)

- **Context Precision:** relevant retrieved chunks / all retrieved chunks
- **Context Recall:** ground-truth claims supported by context / all ground-truth claims
- **Faithfulness:** answer claims supported by context / all answer claims
- **Answer Relevance:** semantic alignment between answer and question

Solved Example 3.2: RAGAS-style scores

Scenario: 8 chunks retrieved, 5 are relevant. 6 gold claims exist, 5 supported by context. Answer makes 7 claims, 6 supported.

Context precision = $5/8 = \mathbf{0.625}$

Context recall = $5/6 = \mathbf{0.833}$

Faithfulness = $6/7 = \mathbf{0.857}$

4 Question Answering Systems

4.1 Question Types and Evolution

Question type	Example	Expected method
Factoid	Where is the Louvre?	Entity or short-span answer
List	Which drugs interact with X?	Retrieve and aggregate multiple items
Definition	What is hyperglycaemia?	Short explanatory answer
Numeric	What was Q3 revenue?	DB or document extraction plus validation
Complex	What is the evidence for Y?	Multidocument synthesis and reasoning
Multi-hop	Who led the project...?	Graph or agentic retrieval

4.2 Retrieval and Ranking Metrics

When a QA system searches for an answer, it returns a ranked list. We use metrics to judge if the correct answer was placed at the very top where the user can see it.

Formula Explained: MRR (Mean Reciprocal Rank)

$$MRR = (1/|Q|) \sum (1/\text{rank of first relevant answer})$$

Plain English: For each query, look at what position (rank) the first correct answer appeared. If it was 1st, score is 1/1. If it was 3rd, score is 1/3. Average these fractions across all queries. It heavily punishes systems that push the right answer down the list.

Solved Example 4.2: MRR Calculation

For two questions, the first relevant answer occurs at rank 2 (for Q1) and rank 1 (for Q2).

Reciprocal Rank for Q1 = $1/2 = 0.5$

Reciprocal Rank for Q2 = $1/1 = 1.0$

$MRR = (0.5 + 1.0)/2 = \mathbf{0.75}$

Formula Explained: nDCG (Normalized Discounted Cumulative Gain)

$$DCG = \sum \frac{rel_i}{\log_2(i+1)}; \quad nDCG = \frac{DCG}{IDCG}$$

Plain English: Used when relevance is a grade (e.g., 3=Perfect, 2=Good, 0=Bad) instead of just yes/no. You add up the grades of the results, but you divide (discount) by a logarithm of their rank position. A "Perfect" result at rank 1 gives lots of points. A "Perfect" result at rank 10 gives very few. IDCG is the maximum possible score if you sorted the results perfectly.

Solved Example 4.4: nDCG Calculation

For graded relevance results: [3, 2, 0, 1] (at ranks 1, 2, 3, 4).

$$DCG = \frac{3}{\log_2(2)} + \frac{2}{\log_2(3)} + \frac{0}{\log_2(4)} + \frac{1}{\log_2(5)} = 3 + 1.262 + 0 + 0.431 = \mathbf{4.693}$$

To find Ideal DCG (IDCG), sort the grades perfectly: [3, 2, 1, 0].

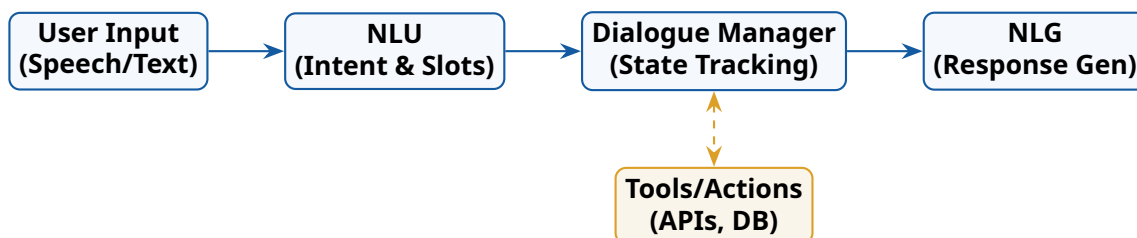
$$IDCG = 3 + 1.262 + \frac{1}{\log_2(4)} + 0 = 3 + 1.262 + 0.5 + 0 = \mathbf{4.762}$$

$$nDCG = 4.693/4.762 = \mathbf{0.986}$$

5 Conversational AI Systems

5.1 Core Components and Evolution

Unlike stateless Question Answering, Conversational AI requires stateful context and memory to manage multi-turn dialogues, clarifications, and real-world actions.



5.2 Intent classification and slot filling

Intent identifies what the user wants. **Slots** capture parameters.

BIO tagging: B marks the first token of an entity, I marks subsequent tokens, and O marks tokens outside entities.

Formula Explained: Slot Error Rate (SER)

$$SER = (S + D + I) / N$$

Plain English: Evaluates the NLU component. Out of N expected slots in a sentence, how many did the system Substitute with the wrong data (S), Delete/miss entirely (D), or completely hallucinate/Insert (I).

Solved Example 5.1: Slot Error Rate

Reference slots expected (N=3): PERSON=Chris, TIME=10:30, LOCATION=Gates 104.

System extraction: PERSON=Chris, TIME=10:00, (LOCATION missing).

Errors:

- PERSON is correct.
- TIME is a Substitution ($S = 1$)
- LOCATION is a Deletion ($D = 1$)
- Insertions ($I = 0$)

$$SER = (1 + 1 + 0) / 3 = 2/3 = \mathbf{0.667} \text{ (or 66.7\% error rate).}$$

6 Sentiment Analysis and Opinion Mining

6.1 Turney algorithm and PMI

Unsupervised sentiment extracts phrases, measures their semantic orientation relative to reference words ("excellent" vs "poor").

Formula Explained: PMI (Pointwise Mutual Information)

$$PMI(x, y) = \log_2[P(x, y)/(P(x)P(y))]$$

$$SO(phrase) = PMI(phrase, "excellent") - PMI(phrase, "poor")$$

Plain English: PMI checks how often two words appear together compared to if they appeared randomly. If "battery" and "terrible" appear together way more often than chance, they have high PMI. Semantic Orientation (SO) simply subtracts the "poor" PMI from the "excellent" PMI to see which way the phrase leans.

Solved Example 6.2: PMI orientation

Suppose $PMI(\text{"fast service", excellent})=2.5$ and $PMI(\text{"fast service", poor})=0.7$.

Semantic orientation = $2.5 - 0.7 = +1.8$ (positive).

If a review has phrase orientations +1.8, +0.6, and -0.4, average orientation = $(1.8 + 0.6 - 0.4)/3 = 2.0/3 = +0.667$, so the review is predicted positive.

7 Privacy, Ethics, Safety, and Governance

7.1 Model fairness metrics

Formula Explained: Fairness Gaps

Demographic Parity Gap: $|P(\hat{y} = 1|GroupA) - P(\hat{y} = 1|GroupB)|$. Do both groups get the same percentage of "Yes" decisions?

Equal Opportunity Gap: $|TPR(GroupA) - TPR(GroupB)|$. Out of the people who actually deserve a "Yes" (True Positives), do both groups get it at the same rate?

Solved Example 7.1: Fairness gaps

Group A receives 60 positive decisions out of 100. Group B receives 45 out of 100.

Demographic parity difference = $0.60 - 0.45 = 0.15$ (15-point gap).

If the True Positive Rate (TPR) for A is 0.80 and B is 0.65.

Equal-opportunity difference = $0.80 - 0.65 = 0.15$.

7.2 Tools and Case Studies

- **Microsoft Presidio / Google DLP:** Used for PII and Privacy masking.
- **IBM AIF360 / Fairlearn:** Used to measure and mitigate group disparities.
- **BertViz / SHAP:** Used for explainability (attention mechanisms).
- **Amazon Recruiting Case:** Trained on historical data that favored men; outcome was a biased model. Lesson: Training history is not a fairness target.

- **Samsung Chatbot Case:** Employees pasted sensitive proprietary code into ChatGPT. Lesson: Requires corporate policy, DLP (Data Loss Prevention) tools, and isolated instances.

8 Master Formula Sheet and Numerical Templates

Concept	Formula	Use
Noisy channel	$\arg \max_w P(x w)P(w)$	Rank spelling candidates
Edit distance	$D(i, j) = \min(\text{delete, insert, substitute})$	Minimum character edits
Add-one smoothing	$(\text{count} + 1) / (\text{context count} + A)$	Avoid zero error probability
Bigram prob	$P(w_1) \prod P(w_i w_{i-1})$	Context-sensitive spelling
Cosine similarity	$(q \cdot d) / (\ q\ \times \ d\)$	Dense retrieval
Precision@k	Relevant in top-k / k	Retrieval purity
Recall@k	Relevant in top-k / all relevant	Retrieval coverage
F1	$2PR / (P + R)$	Balance precision and recall
MRR	mean(1/first relevant rank)	First-answer ranking
DCG	$\sum rel_i / \log_2(i + 1)$	Graded ranking
nDCG	$DCG / IDC G$	Normalized graded ranking
ROUGE-L F1	$2P_{lcs}R_{lcs} / (P_{lcs} + R_{lcs})$	Sequence overlap
Context precision	Relevant retrieved chunks / all chunks	Retriever precision
Slot Error Rate	$(S + D + I) / N$	Dialogue slot quality
PMI	$\log_2[P(x, y) / (P(x)P(y))]$	Association strength
F0.5	$1.25PR / (0.25P + R)$	GEC precision-weighted score
Demo. parity gap	$P(\hat{y} = 1 A = 0) - P(\hat{y} = 1 A = 1)$	Group outcome difference

9 Fully Solved Previous Mid-Semester Paper

Question 1: Spell checking

1(a) Error identification

- commitee → committee (Non-word; insertion of missing t or repeated-letter error)
- recieved → received (Non-word; adjacent transposition of i and e)
- excellant → excellent (Non-word; substitution a → e)
- participents → participants (Non-word; substitution e → a)
- survey → survey (Non-word; substitution a → e)

1(b) Noisy-channel correction of "recieved"

Candidate	P(w)	P(x w)	Product
received	0.0000247	0.00040000	9.880e-09
receive	0.0000197	0.00020000	3.940e-09
receiver	0.0000091	0.00006000	5.460e-10
deceived	0.0000043	0.00003000	1.290e-10

Answer: 'received' has the maximum unnormalized posterior, 9.88×10^{-9} so it is selected.

Language	Tickets	NER acc.	Correct NER	TC acc.	Correct TC
English	$8000 \times 0.55 = 4400$	82%	$4400 \times 0.82 = 3608$	90%	3960
German	$8000 \times 0.30 = 2400$	76%	1824	83%	1992
Italian	$8000 \times 0.15 = 1200$	70%	840	78%	936
Total	8000		6272		6888

Question 2(a): Multilingual weighted accuracy

Total correct NER = 6272 (78.4%). Overall TC accuracy = $6888/8000 = 86.1\%$.

To reach 87%, required correct = $0.87 \times 8000 = 6960$.

Additional correct = $6960 - 6888 = 72$ (0.9 percentage-point improvement).

Question 3(a): QA ranking and top-1 classification

System A: Q1 rank/RR = 2/0.5, Q2 rank/RR = 2/0.5. $MRR = (0.5 + 0.5)/2 = 0.50$.

System B: Q1 rank/RR = 2/0.5, Q2 rank/RR = 1/1.0. $MRR = (0.5 + 1)/2 = 0.75$.

Recommendation: System B due to higher MRR and top-1 F1.

Slot Error Rate and BIO tags

Please/O book/O an/O urgent/O appointment/O for/O Priya/B-PATIENT_NAME Kapoor/I-PATIENT_NAME
with/O Dr./O Rao/O for/O severe/B-SYMPTOM abdominal/I-SYMPTOM pain/I-SYMPTOM on/O 15th/B-
APPOINTMENT_DATE June/I-APPOINTMENT_DATE 2025/I-APPOINTMENT_DATE at/O 2:00/B-APPOINTMENT_TIME
pm/I-APPOINTMENT_TIME ./O

Question 4: Renewable-energy knowledge graph

- **Entities:** Region, Plant, SolarPlant, WindFarm, Turbine, Operator, MeterReading.
- **Relations:** LOCATED_IN, OPERATED_BY, HAS_TURBINE, HAS_READING.
- **Data Types:** Static data (identity, location, capacity). Dynamic data (power output, weather). Active data (alerts, outages).

10 High-Probability Practice Questions with Solutions (6-Mark Format)

Practice 1: Spell Check & Grammar Evaluation (6 Marks)

Part A (3 Marks): A spelling checker suggests "definitely" and "defiantly" for the non-word error "definatly". Given priors $P(\text{definitely}) = 0.0001$, $P(\text{defiantly}) = 0.00005$ and error likelihoods $P(\text{definatly}|\text{definitely}) = 0.05$, $P(\text{definatly}|\text{defiantly}) = 0.01$, calculate the unnormalized posteriors using the Noisy Channel Model and select the winner.

Solution A: Formula: $\hat{w} = \arg \max P(x|w) \times P(w)$

Candidate 1 ("definitely"): $0.05 \times 0.0001 = \mathbf{0.000005}$

Candidate 2 ("defiantly"): $0.01 \times 0.00005 = \mathbf{0.0000005}$

Since $0.000005 > 0.0000005$, the system selects "**definitely**".

Part B (3 Marks): Calculate the $F_{0.5}$ score for a grammar checker that made 120 suggestions, 90 of which were correct, against a gold standard of 150 actual errors. Explain why $\beta = 0.5$ is used.

Solution B: Precision (P) = $90/120 = 0.75$. Recall (R) = $90/150 = 0.60$.

$F_{0.5} = (1 + 0.5^2) \frac{0.75 \times 0.60}{(0.5^2 \times 0.75) + 0.60} = 1.25 \times \frac{0.45}{0.1875 + 0.60} = \frac{0.5625}{0.7875} = \mathbf{0.714}$.

Explanation: $\beta = 0.5$ is used because false positive corrections (changing correct grammar to incorrect) highly annoy users. It weights Precision twice as heavily as Recall.

Practice 2: RAG Pipeline Design & Evaluation (6 Marks)

Part A (3 Marks): Design an Agentic RAG pipeline for a financial institution to compare Q3 and Q4 revenues from SEC filings. State the components and flow.

Solution A: 1. **LLM Orchestrator:** Receives "Compare Q3/Q4 revenues". Decomposes the task.

2. **Retrieval Tool:** Queries vector DB containing chunked SEC filings for Q3, then Q4.

3. **Extraction & Math Tool:** Extracts numeric values and calculates the difference.

4. **Verification & NLG:** Cross-checks values, formats response, and cites sources.

Part B (3 Marks): Evaluate RAG metrics: 8 chunks retrieved, 5 relevant. 6 gold claims exist, 5 supported by context. Answer makes 7 claims, 6 supported. Calculate Context Precision, Context Recall, and Faithfulness.

Solution B: Context precision = $5/8 = \mathbf{0.625}$

Context recall = $5/6 = \mathbf{0.833}$

Faithfulness = $6/7 = \mathbf{0.857}$

Practice 3: Retrieval Metrics (MRR & nDCG) (6 Marks)

Part A (3 Marks): For a set of 3 queries, the first relevant documents are retrieved at ranks 1, 3, and 2 respectively. Calculate the Mean Reciprocal Rank (MRR).

Solution A: $MRR = \frac{1}{|Q|} \sum \frac{1}{\text{rank}_i}$

$MRR = \frac{1}{3} \left(\frac{1}{1} + \frac{1}{3} + \frac{1}{2} \right) = \frac{1}{3} (1 + 0.333 + 0.5) = \frac{1.833}{3} = \mathbf{0.611}$

Part B (3 Marks): Calculate nDCG for graded relevance results: [3, 1, 0, 2] (at ranks 1, 2, 3, 4).

Solution B: $DCG = \frac{3}{\log_2(2)} + \frac{1}{\log_2(3)} + \frac{0}{\log_2(4)} + \frac{2}{\log_2(5)} = 3 + 0.631 + 0 + 0.861 = \mathbf{4.492}$

Ideal Sort: [3, 2, 1, 0].

$IDCG = 3 + \frac{2}{\log_2(3)} + \frac{1}{\log_2(4)} + 0 = 3 + 1.262 + 0.5 = \mathbf{4.762}$

$$nDCG = 4.492/4.762 = \mathbf{0.943}$$

Practice 4: Conversational AI & Sentiment (6 Marks)

Part A (3 Marks): A dialogue system is expected to extract 4 slots: [DESTINATION, DATE, TIME, CLASS]. It extracts DESTINATION correctly, misses DATE entirely, substitutes TIME with the wrong format, and hallucinates an AIRLINE slot. Calculate Slot Error Rate (SER).

Solution A: Total expected slots (N) = 4.

Errors: Deletion (D) = 1 (DATE). Substitution (S) = 1 (TIME). Insertion (I) = 1 (AIRLINE).

$$SER = (S + D + I)/N = (1 + 1 + 1)/4 = 3/4 = \mathbf{0.75} \text{ (75\% error rate).}$$

Part B (3 Marks): Calculate the Semantic Orientation (SO) of the phrase "fast processor" if $PMI(\text{"fast processor", "excellent"}) = 3.2$ and $PMI(\text{"fast processor", "poor"}) = 1.1$. Determine the overall polarity.

Solution B: $SO(\text{phrase}) = PMI(\text{phrase, "excellent"}) - PMI(\text{phrase, "poor"})$

$$SO = 3.2 - 1.1 = \mathbf{+2.1}$$

The positive SO score indicates the polarity of the phrase is **Positive**.

Practice 5: Ethics, Fairness & Privacy (6 Marks)

Part A (3 Marks): A loan approval model approves 75 out of 120 applicants from Group A, and 40 out of 80 applicants from Group B. The True Positive Rates are 85% for Group A and 70% for Group B. Calculate the Demographic Parity Gap and the Equal Opportunity Gap.

Solution A: Approval Rate A = $75/120 = 0.625$. Approval Rate B = $40/80 = 0.500$.

Demographic Parity Gap = $|0.625 - 0.500| = \mathbf{0.125}$ (12.5 points).

Equal Opportunity Gap = $|0.85 - 0.70| = \mathbf{0.15}$ (15 points).

Part B (3 Marks): An NLP healthcare chatbot is built using an open-source LLM. What are three critical deployment controls required to ensure privacy and safety?

Solution B: 1. **PII Scrubbing:** Implement Microsoft Presidio or Google DLP to mask patient names and identifiers before the prompt hits the LLM.

2. **Output Guardrails:** Use tools like NeMo Guardrails to prevent the LLM from hallucinating medical advice or dosage limits.

3. **Audit Logging:** Log model latency, errors, and system versioning, but strictly exclude unanonymized patient prompts from the storage logs.

11 Final Revision Checklist

- ✓ I can compute noisy-channel candidate scores and explain every term.
- ✓ I can distinguish non-word, real-word, syntactic, semantic, and punctuation errors.
- ✓ I can compare naive, graph, agentic, and multimodal RAG.
- ✓ I can calculate P@k, R@k, F1, MRR, MAP, nDCG, ROUGE-L, and RAGAS-style scores.
- ✓ I can compare IR, knowledge-based, extractive, generative, RAG, and agentic QA.
- ✓ I can design an NLU-slot-dialogue-RAG-tool-NLG conversational architecture.
- ✓ I can calculate SER and write BIO tags.

- ✓ I can explain lexicon, PMI, ABSA, ML, and transformer sentiment methods.
- ✓ I can calculate classification metrics and fairness gaps.
- ✓ I can discuss privacy controls at data, model, evaluation, deployment, and monitoring stages.
- ✓ I can answer a 6-mark design question with assumptions, architecture, updating, metrics, and governance.